

Supplementary Material

CoM-VLA: Coordinated Multi-Robot Vision-Language-Action Model for Decentralized Formation Reconfiguration

Author Names Omitted for Anonymous Review. Paper-ID [1881]

I. IMPLEMENTATION DETAILS

A. Vision-Grounded Multi-Robot Formation Instruction

As described in the main paper, given the estimated navigable width $w(\mathbf{I}_i)$, the feasible formation candidates are selected as

$$\mathcal{F}_i^K = \text{TopK}(\{f \in \mathcal{F} \mid s_f + 2\delta \leq w(\mathbf{I}_i)\}, s_f), \quad (1)$$

where $\mathcal{F} = \{\text{line}, \text{wedge}, \text{column}\}$ and $K = 2$. For the five-robot team used in the main experiments, the lateral formation spans are $s_{\text{line}} = 2.16$ m, $s_{\text{wedge}} = 1.44$ m, and $s_{\text{column}} = 0$ m. We set $\delta = 0.24$ m on each side, accounting for half of the 0.28 m robot width and an additional 0.10 m clearance. This gives feasibility thresholds of 2.64 m, 1.92 m, and 0.48 m for the line, wedge, and column formations, respectively.

The selected candidates are incorporated into a prompt and reported to the operator. The final selected formation is denoted by τ and used as the formation instruction for the VLA policy. In implementation, the navigable width is estimated from a single 224×224 RGB observation using the known camera intrinsics and fixed camera mounting geometry. We identify the visible floor region and project its lateral boundaries onto the ground plane to obtain metric corridor widths. The width is evaluated at $N_d = 12$ uniformly sampled look-ahead distances over $[2.5, 6.0]$ m, and the minimum valid width is used as the conservative estimate $w(\mathbf{I}_i)$. The width estimate is updated at every control step.

B. Instruction-Grounded Team Coordination

To enable team-level coordination, we introduce an instruction-grounded graph network. For each robot i , its local state $\mathbf{s}_i \in \mathbb{R}^4$ is concatenated with the formation instruction $\tau \in \mathbb{R}^3$, represented as a one-hot vector. A shared two-layer encoder $\mathbf{W}^z$ with hidden dimension 64 maps the resulting 7-dimensional input to a latent representation

$$\mathbf{z}_i = \mathbf{W}^z[\mathbf{s}_i \parallel \tau], \quad \mathbf{z}_i \in \mathbb{R}^{32}. \quad (2)$$

For each ordered robot pair (i, j) within the communication radius, a shared two-layer edge predictor $\mathbf{W}^p$ with the same hidden dimension takes $[\mathbf{z}_i \parallel \mathbf{d}_{ij}] \in \mathbb{R}^{34}$ and predicts the communication probability

$$p_{ij} = \sigma(\mathbf{W}^p[\mathbf{z}_i \parallel \mathbf{d}_{ij}]), \quad (3)$$

where $\mathbf{d}_{ij} \in \mathbb{R}^2$ denotes the relative position of robot j in robot i 's body frame.

During training, binary communication edges are sampled using a straight-through Gumbel-Sigmoid estimator, while during execution they are obtained by thresholding p_{ij} at $\delta_c = 0.5$. The resulting topology is used for message passing,

$$\mathbf{h}_i = \mathbf{W}^s \mathbf{z}_i + \sum_{j \neq i} A_{ij} \mathbf{W}^m \mathbf{z}_j, \quad (4)$$

where $\mathbf{W}^s$ and $\mathbf{W}^m$ map the 32-dimensional latent representations to an 8-dimensional team-aware embedding $\mathbf{h}_i$. All mappings are shared across robots and robot pairs, so the network contains no parameters that depend on the team size n .

To train the adaptive communication topology, we regularize both communication sparsity and temporal consistency as

$$\mathcal{L}_{\text{comm}} = \overbrace{\frac{\lambda_c}{n(n-1)} \sum_{i=1}^n \sum_{j \neq i} p_{ij}^t}^{\text{Communication sparsity}} + \overbrace{\frac{\lambda_t}{n(n-1)} \sum_{i=1}^n \sum_{j \neq i} |p_{ij}^t - p_{ij}^{t-1}|}^{\text{Temporal consistency}}, \quad (5)$$

where the first term encourages sparse communication and the second promotes temporal consistency of the learned topology. We set $\lambda_c = 0.02$ and $\lambda_t = 0.05$ for all experiments.

C. Decentralized Multi-Robot VLA Policy

We build CoM-VLA upon the pretrained OmniVLA navigation backbone [1], which employs fused DINOv2 [2] and SigLIP [3] visual encoders together with a Llama-2 7B language model [4], and extend the backbone to team-level coordination. The team-aware embedding $\mathbf{h}_t$ is projected into the VLA embedding space and appended as a single token to condition decentralized action generation.

Following OmniVLA, the policy predicts a short-horizon sequence of 8 navigation waypoints, which are discretized into action tokens. During execution, only the first predicted waypoint is converted into the robot action $\mathbf{a}_t \in \mathbb{R}^2$, and the policy is re-evaluated at every control step, resulting in closed-loop receding-horizon execution. The VLA policy is queried at 20 Hz on an NVIDIA RTX 4090 GPU, and each predicted action is applied for one control step.

II. TRAINING DETAILS

A. Training Setup

We construct 35 maze layouts in total, using 25 corridor mazes for training and holding out 10 unseen layouts for evaluation. The training demonstrations comprise 163 formation transitions.

A single shared VLA policy is applied independently to each robot using its local visual observation, formation instruction, and team-aware embedding. The same policy parameters are shared across all robots and do not depend on the team size n , allowing the learned policy to be applied to different team sizes without architectural modification.

The pretrained vision and language backbone weights are frozen and adapted using LoRA with rank $r = 32$, scaling $\alpha = 16$, and zero dropout. LoRA is applied to the linear layers of the pretrained VLA, resulting in 110.7M trainable adapter parameters. The coordination network and team-embedding projector are trained in full together with the LoRA parameters. Overall, 127.5M of the 7.67B parameters are trainable (1.66%).

The policy is optimized using cross-entropy loss over the discretized expert action tokens together with the communication objective $\mathcal{L}_{\text{comm}}$. We use AdamW with a learning rate of 5×10^{-5} , zero weight decay, and 200 linear warm-up steps. Each training batch contains one robot team, corresponding to n robot-level samples, and gradients are accumulated over four steps before each optimizer update. Training is performed for 240,000 training steps, corresponding to approximately 13 epochs, on a single NVIDIA H100 GPU with 80 GB of memory.

B. Expert Demonstration Generation

We generate expert demonstrations using a centralized nonlinear model predictive control (MPC) planner with access to global robot states and prior environmental information. Given the reference trajectories of all robots, the expert jointly optimizes their actions by minimizing

$$\mathcal{J} = \sum_{k=1}^T \sum_{i=1}^n \left(w_p \|\mathbf{p}_k^i - \mathbf{r}_k^i\|^2 + w_u \|\mathbf{u}_k^i\|^2 + w_\Delta \|\mathbf{u}_k^i - \mathbf{u}_{k-1}^i\|^2 \right), \quad (6)$$

where n is the number of robots, T is the planning horizon, $\mathbf{p}_k^i \in \mathbb{R}^2$ denotes the planar position of robot i at step k , $\mathbf{r}_k^i \in \mathbb{R}^2$ denotes its reference position, and $\mathbf{u}_k^i \in \mathbb{R}^2$ denotes the planar control action. The weights w_p , w_u , and w_Δ balance reference tracking, control effort, and temporal smoothness, respectively.

The optimization is formulated using CasADi [5] and solved with IPOPT [6]. The resulting trajectories are replayed in simulation to collect the visual observations, formation instructions, robot states, and expert actions used for imitation learning.

The MPC expert is centralized and privileged, as it jointly plans for all robots using global robot states and prior environmental information. In contrast, CoM-VLA does not receive the prior environment map during execution. Each robot instead generates its action from its onboard visual observation, formation instruction, and team-aware representation.

III. ADDITIONAL EXPERIMENTS

A. Ablation of Communication Topology

We further ablate the two terms in the communication objective in Eq. 5 to analyze their effects on the learned communication topology. We compare the full objective ($\lambda_c = 0.02$, $\lambda_t = 0.05$) against variants without temporal consistency ($\lambda_t = 0$) and without communication sparsity ($\lambda_c = 0$). Each variant is trained with three random seeds and evaluated under the same protocol.

We quantitatively evaluate the learned communication topology using four metrics: (1) *Active Edges*, the average number of active directed communication edges; (2) *Topology Hamming Distance*, the average number of directed edges that differ between communication topologies under different formation conditions; (3) *Hold Variation*, denoted as $|\Delta p|_{\text{hold}}$, the mean absolute change in communication probabilities between consecutive control steps while maintaining the same formation; and (4) *Switch Variation*, denoted as $|\Delta p|_{\text{switch}}$, the mean absolute change in communication probabilities during formation reconfiguration.

The qualitative results are shown in Figure 1. Without the communication sparsity term, the learned topology becomes fully connected and remains nearly unchanged throughout the formation reconfiguration. In contrast, removing the temporal-consistency term preserves sparse and formation-dependent communication, but the communication probabilities exhibit larger fluctuations during formation maintenance.

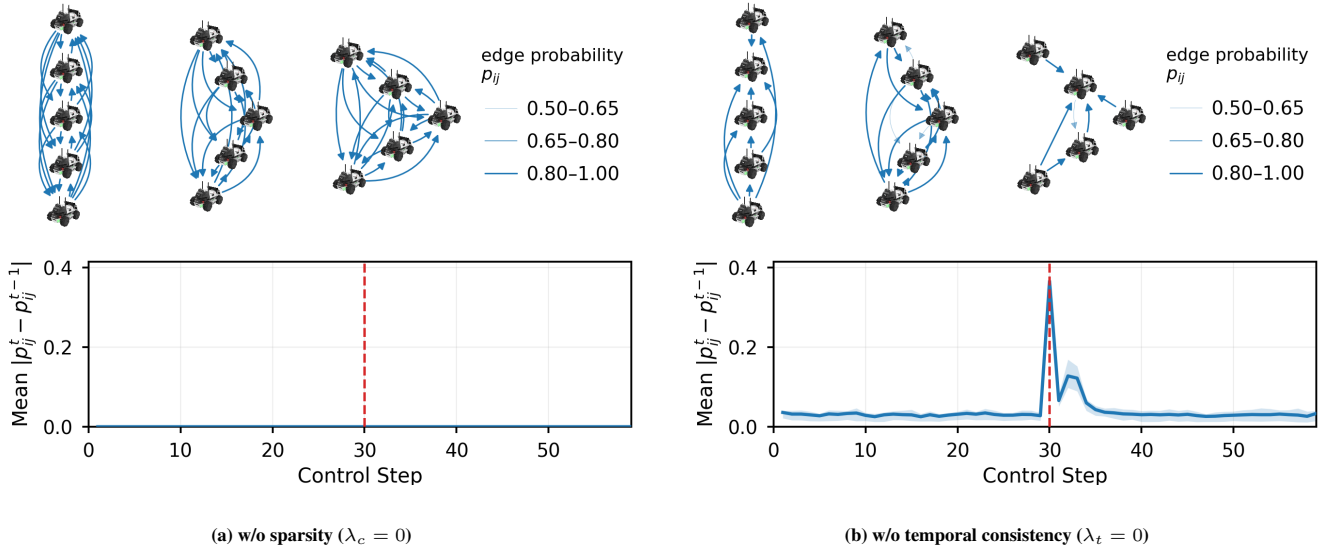


Fig. 1. Qualitative ablation during line-to-wedge reconfiguration. Without sparsity, the topology becomes fully connected; without temporal consistency, communication probabilities fluctuate more during formation maintenance. The red dashed line marks the formation transition.

TABLE I
ABLATION OF THE COMMUNICATION OBJECTIVE.

Method	Active Edges ↓	Hamming Dist. ↑	Hold Variation ↓	Switch Variation
Full	8.02 ± 0.20	4.56 ± 0.35	4.4×10^{-3}	0.114
w/o temporal consistency	7.17 ± 0.58	2.96 ± 0.17	1.39×10^{-2}	0.147
w/o communication sparsity	20.00 ± 0.00	0.00 ± 0.00	≈ 0	≈ 0

The quantitative results are shown in Table I. Removing the communication sparsity term activates all 20 directed communication edges and yields zero Hamming distance, confirming that the topology becomes identical across formations. Its near-zero hold and switch variations result from the graph remaining constantly fully connected rather than from meaningful temporal consistency.

Removing the temporal-consistency term maintains sparse communication, with 7.17 active edges on average, but increases $|\Delta p|_{\text{hold}}$ from 4.4×10^{-3} to 1.39×10^{-2} , corresponding to approximately a $3.2 \times$ increase in temporal variation during formation maintenance. Its Hamming distance also decreases from 4.56 to 2.96, indicating less distinct communication topologies across formation conditions.

These results show that the two objective terms play complementary roles: λ_c encourages sparse and formation-dependent communication, while λ_t promotes temporal stability during formation maintenance while still allowing the topology to adapt during formation reconfiguration.

REFERENCES

- [1] N. Hirose, C. Glossop, D. Shah, and S. Levine, “Omnivla: An omni-modal vision-language-action model for robot navigation,” *arXiv*, 2025.
- [2] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, “Dinov2: Learning robust visual features without supervision,” *arXiv*, 2023.
- [3] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *IEEE/CVF International Conference on Computer Vision*, 2023, pp. 11 941–11 952.
- [4] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv*, 2023.
- [5] J. A. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “Casadi: a software framework for nonlinear optimization and optimal control: Jae andersson *et al.*,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [6] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.